231501173

Ex 4: Comparative Analysis of Large Language Models

Learning Objective:
Develop a Python program that performs a comparative analysis of Large Language Models by analyzing the text-generation performance of OpenAI, Gemini, and Hugging Face models.

Steps:

1. Import Required Libraries (OpenAI, HuggingFace Transformers, Gemini API)
2. Configure OpenAI key and Google Gemini API key. (This key should be personal. Do not share with anyone).
3. Initialize the Models (OpenAI, Gemini, and Hugging Face).
4. Write function to send prompt and receive response from OpenAI, Gemini, and Hugging Face models
5. Provide a common text prompt. (e.g., “Write a haiku about AI and nature”).
6. Generate Outputs by sending the same prompt to all three models and collect their responses.
7. Display the Model Responses individually. Use Markdown / HTML formatting to display results clearly.
8. Construct an evaluator. (For evaluator you can use Gemini) and produce ranks.
9. Display the ranked model names in best → worst order.

CODE:

```
import google.generativeai as genai
from transformers import pipeline
from huggingface_hub import login
import time
ge na i. co nfigure (ap i_ke y= "AIzaS yBQ Ka35 UnFwV yCNG uo6k nG
lqberp TP xKz M")
login("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")
gemini_model = genai.GenerativeModel("gemini-2.5-flash")
tiny_model = pipeline(
"text-generation",
model="TinyLlama/TinyLlama-1.1B-Chat-v1.0"
)
hf_model = pipeline(
    "text-generation",
    model="distilgpt2"
)
def query_gemini(prompt):
```

```
 start = time.time()
    res = gemini_model.generate_content(prompt)
    return res.text, time.time() - start

def query_tinyllama(prompt):
    start = time.time()
    res = tiny_model(prompt, max_length=80,
do_sample=True,temperature=0.7)
    return res[0]['generated_text'], time.time() - start

def query_hf(prompt):
    start = time.time()
    res = hf_model(prompt, max_length=80, do_sample=True,temperature=0.7)
    return res[0]['generated_text'], time.time() - start
prompt = "Write a haiku about AI and nature"
gemini_out, gemini_time = query_gemini(prompt)
tiny_out, tiny_time = query_tinyllama(prompt)
hf_out, hf_time = query_hf(prompt)
from IPython.display import Markdown, display
display(Markdown(f"""
## Gemini
Time: {gemini_time:.2f}s
{gemini_out}
---
## TinyLlama (HF)
Time: {tiny_time:.2f}s
{tiny_out}
---
## Hugging Face (DistilGPT2)
Time: {hf_time:.2f}s
{hf_out}
"""))
eval_prompt = f"""
Evaluate the outputs based on:
- Relevance
- Creativity
- Clarity
Prompt: {prompt}
Gemini:
{gemini_out}
TinyLlama:
{tiny_out}
HuggingFace:
```

```
{hf_out}
Return ranking BEST to WORST.
"""
ranking = gemini_model.generate_content(eval_prompt).text
display(
    n(f"""
# Final Ranking

{ranking}
"""))
```

OUTPUT:

```
Loading weights: 100% ||||||||||||||||||||||||||||||||||||||||||||||||||| 201/201 [00:00<00:00, 419.84it/s, Materializing param=model.norm.weight]
Loading weights: 100% ||||||||||||||||||||||||||||||||||||||||||||||||||| 76/76 [00:00<00:00, 316.98it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: distilgpt2
Key                                     | Status     |  |
----------------------------------------+------------+--+-
transformer.h.{0, 1, 2, 3, 4, 5}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```

Final Ranking

Here's the evaluation of the outputs:

1. Gemini

- **Relevance:** 5/5 - Perfectly addresses the prompt. It is a haiku, and it clearly links AI ("Circuits hum a tune") with nature ("Wind whispers through ancient leaves") and their interaction ("Two worlds gently meet"). The syllable count (5-7-5) is correct.
- **Creativity:** 4/5 - The imagery is evocative, subtly contrasting the mechanical hum with the natural whisper, leading to a gentle convergence. It avoids clichés and offers a peaceful perspective on their meeting.
- **Clarity:** 5/5 - The language is simple, direct, and the meaning is immediately clear. The haiku flows well and is easy to understand.

2. TinyLlama

- **Relevance:** 1/5 - This output does not answer the prompt at all. Instead of writing a haiku, it provides a rephrased and expanded set of instructions for writing a haiku. It completely missed the task.
- **Creativity:** 0/5 - There is no creative output in the form of a haiku to evaluate. The text provided is an instruction set.
- **Clarity:** 3/5 - The instructions themselves are clear, but the clarity of the *output as an answer to the prompt* is non-existent, as it fails to provide the requested content.

3. HuggingFace

- **Relevance:** 0/5 - This output does not answer the prompt. It repeats the prompt and then adds a completely nonsensical and irrelevant sentence.
- **Creativity:** 0/5 - There is no creative output. The addition is gibberish.
- **Clarity:** 1/5 - The first part (repeating the prompt) is clear, but the second part is utterly unclear and makes no sense in context.

Return ranking BEST to WORST:

1. **Gemini** (Best - Fully met all aspects of the prompt with quality)
2. **TinyLlama** (Worst - Failed to write a haiku, instead rephrasing the prompt. While a clear instruction, it's not an answer.)
3. **HuggingFace** (Absolute Worst - Failed to write a haiku and responded with gibberish after repeating the prompt.)

Learning Outcome:

Students will be able to integrate and interact with OpenAI, Gemini, and Hugging Face APIs and evaluate model outputs based on relevance.